

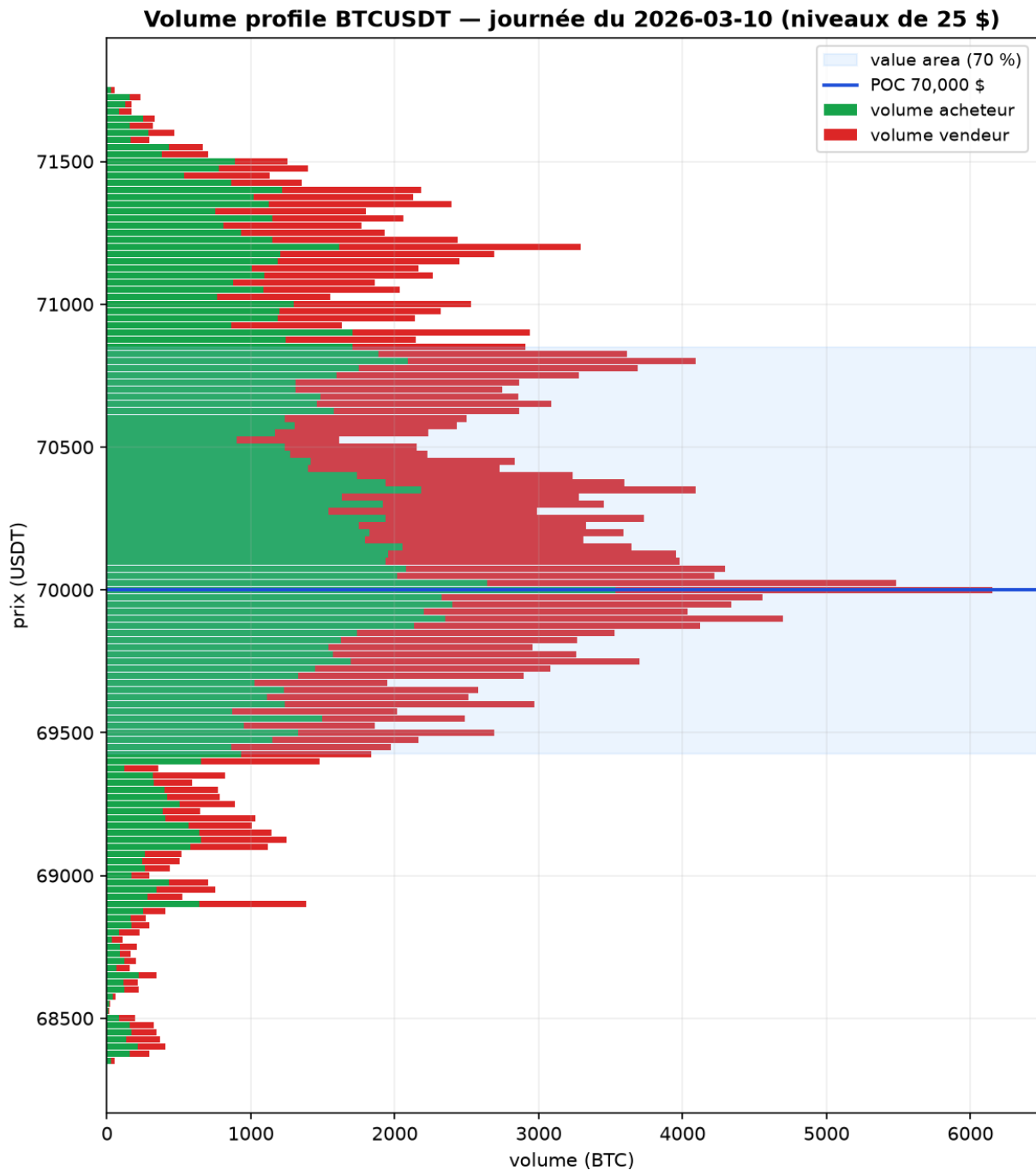
Extension — Le volume profile par session : la dimension que l'OHLCV a écrasée

Toutes nos stratégies jusqu'ici ne regardent que le **prix**. Le volume, pourtant présent dans chaque barre, n'a jamais servi. Cette leçon l'exploite deux fois. D'abord l'**analyse des volumes** : notre CSV sépare déjà `buy_volume` (volume dont l'agresseur est acheteur) du volume total — leur différence, le **delta**, dit *qui* pousse le marché, pas seulement où il va. Ensuite le **volume profile** : la distribution du volume **par niveau de prix**. Et là, problème — une barre OHLCV a écrasé cette information. Elle sait que 1 571 BTC ont changé de mains entre 87 600 et 87 819 \$, pas à *quel prix*. Pour la retrouver, on retourne aux **ticks** — les 328 M de trades bruts dont nos barres sont issues (Moteur 2 : les barres ne sont qu'une agrégation).

Le profil : POC et value area

Empilez le volume de chaque trade dans une case de 25 \$ selon son prix : l'histogramme obtenu est le volume profile. Trois niveaux le résument :

- **POC** (*Point of Control*) : le prix le plus traité — là où acheteurs et vendeurs se sont le plus mis d'accord ;
- **value area** (VAL → VAH) : la plus petite plage contiguë autour du POC contenant **70 %** du volume — la zone de « juste prix » de la période ;
- hors value area : des prix que le marché a **explorés sans les accepter**.



Volume profile de la journée du 10 mars 2026 : histogramme horizontal du volume par niveau de 25 \$, volume acheteur en vert et vendeur en rouge, en forme de cloche autour du POC à 70 000 \$, avec la value area surlignée d'environ 69 450 à 70 850 \$.

Figure — Une période de marché vue par les volumes : la cloche se centre sur le POC — ici pile 70 000 \$, le chiffre rond psychologique. (Script : le générateur ci-dessous, option --profil-jour.)

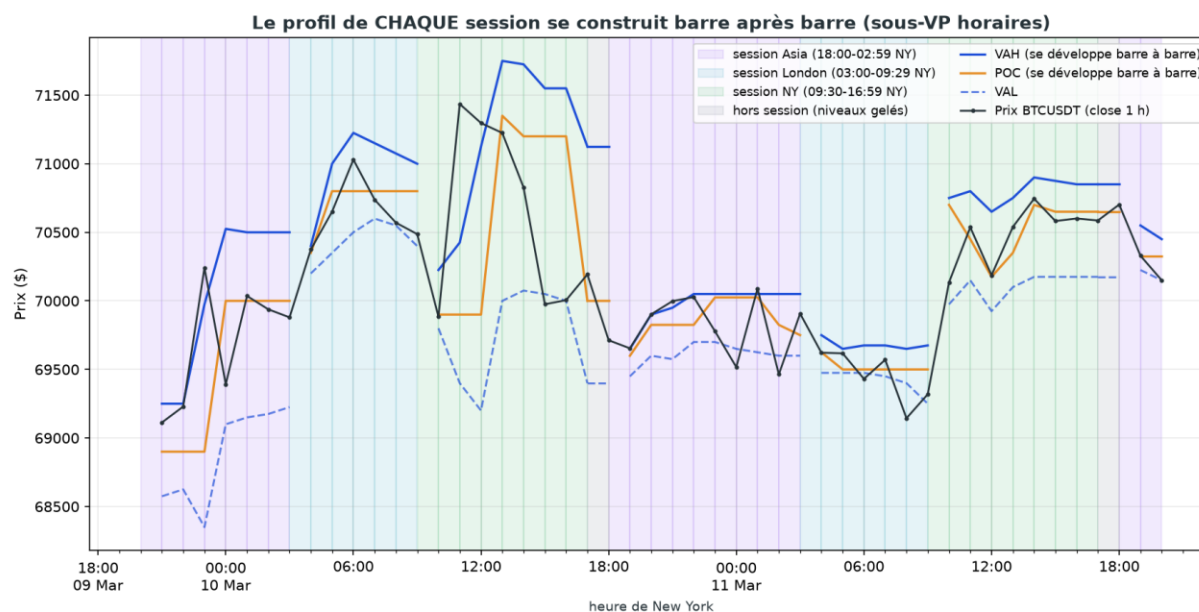
La grammaire de lecture est celle des enchères : un prix qui **rentre** dans la valeur area tend à traverser vers le POC (retour au juste prix) ; un prix qui **sort** de la valeur area et y reste est **accepté** — le marché déménage. Deux lectures opposées du même objet, qu'on mesurera toutes les deux.

L'ancrage : un profil de QUOI, exactement ?

Un profil n'existe que par sa **période d'agrégation** — son *ancrage* — et le choix n'est pas anodin. La figure ci-dessus agrège une « journée » : mais laquelle ? 00:00–23:59 UTC est un ancrage **arbitraire** pour un actif 24/7 — il coupe et recolle des enchères qui n'ont rien à voir. Les ancres qui correspondent à la structure réelle du marché sont les **sessions**, définies en **heure de New York** (donc mobiles en UTC au passage à l'heure d'été — le piège du 8 mars, encore lui) :

Session	Heures NY	En UTC (hiver / été)
Asia	18:00 → 02:59	23:00 → 07:59 / 22:00 → 06:59
London	03:00 → 09:29	08:00 → 14:29 / 07:00 → 13:29
NY	09:30 → 16:59	14:30 → 21:59 / 13:30 → 20:59
(hors session)	17:00 → 17:59	niveaux gelés de la dernière session

Chaque session est une **enchère autonome** : son profil repart de zéro à l'ouverture, puis se **développe** barre 1H après barre 1H — ce sont les *sous-VP* intra-session : à chaque clôture on connaît l'état courant du POC/VAH/VAL de la session, et on les voit se déplacer à mesure que l'enchère se construit.



Deux jours de prix horaire avec les bandes de session Asia, London et NY colorées ; dans chaque bande, les niveaux VAH, POC et VAL repartent de zéro à l'ouverture, bougent nerveusement sur les premières barres puis se stabilisent ; dans les zones grises hors session, les niveaux restent figés.

Figure — Le profil de chaque session se construit sous nos yeux : nerveux à l'ouverture (peu de volume accumulé), stable en fin de session. Hors session (gris), les niveaux restent gelés. (Script : `fig_volume_profile_sessions.py`)

Reconstruire les profils depuis les ticks

Le générateur `volume_profile_features.py` refait le chemin de la Moteur 2 (ticks → agrégat), en agrégeant par **prix** et par **session**. Détail d'implémentation qui compte : 09:30 NY tombe en **milieu de barre 1H UTC** (13:30 ou 14:30 UTC selon la saison) → les ticks sont d'abord agrégés en sous-briques de **30 minutes**, dont les frontières coïncident exactement avec toutes les bornes de session, été comme hiver.

```
python backtests/volume_profile_features.py --profil-jour 2026-03-10
streaming des ticks (une passe, sous-briques de 30 min)...
  8688 sous-briques de 30 min, 131699 cellules (niveau, brique)
features -> H:/Crypto/historique/ohlcv/BTCUSDT-um/features_vp.csv (zone sessions,
tick 25 $)
```

Le contrat du CSV produit mérite une lecture attentive — c'est là que la causalité se joue (trois lignes types d'une même journée, réordonnées pour le récit ; le fichier, lui, est strictement chronologique) :

```
time,session,barres,delta,poc,vah,val
2026-01-15 23:00:00+00:00,asia,1,...      <- 18:00 NY : l'enchère Asia démarre
(profil neuf)
2026-01-15 14:00:00+00:00,ny,1,...        <- la barre qui contient 09:30 NY (14:30
UTC l'hiver)
2026-01-15 22:00:00+00:00,hors,0,...      <- 17:00-17:59 NY : niveaux GELÉS de la
session NY
```

La ligne `t` porte le profil de la session en cours, **accumulé de l'ouverture de session à la clôture de la barre `t` incluse** (barres = ancienneté du profil). Elle n'est donc **connaissable qu'à `t+1h`** — même contrat d'horodatage que `1H.csv` (Moteur 2), et LEAN va le faire respecter tout seul : `end_time = t+1h` → la ligne est livrée **dans le même Slice** que la barre de prix qu'elle résume. Causal par construction — on y reviendra pour le falsifier. Point de contrôle DST mesuré : la première barre `ny` passe de **14:00 UTC** (janvier, EST) à **13:00 UTC** (après le 8 mars, EDT).

Deux flux custom dans le même algorithme

Nouveauté LEAN de cette leçon : l'algo `volume_profile.py` consomme **deux** `PythonData` — le lecteur prix habituel, plus un lecteur de features :

```
class VpFeatures(PythonData):
    """La ligne t porte le profil de la session en cours, développé de l'ouverture
de
    session à la clôture de la barre t (incluse). end_time = t+1h -> livrée à la
    clôture de la barre t, dans le MEME Slice que le prix. Causal."""

    def get_source(self, config, date, is_live):
        return SubscriptionDataSource(VP_FILE,
SubscriptionTransportMedium.LOCAL_FILE)

    def reader(self, config, line, date, is_live):
        if not line or not line[0].isdigit():
            return None
        cols = line.split(",")
        if cols[4] == "":
            # tout début d'historique : aucun profil
encore
            return None
        bar = VpFeatures()
        bar.symbol = config.symbol
        t_open = datetime.strptime(cols[0][:19], "%Y-%m-%d %H:%M:%S")
        bar.time = t_open
```

```

        bar.end_time = t_open + timedelta(hours=1)
        bar.value = float(cols[4])          # value obligatoire (jamais de NaN) : le
POC
        bar["session"] = cols[1]            # asia | london | ny | hors
        bar["barres"] = float(cols[2])      # ancienneté du profil dans la session
        bar["delta"] = float(cols[3])
        bar["poc"] = float(cols[4])
        bar["vah"] = float(cols[5])
        bar["val"] = float(cols[6])
        return bar

```

Dans `initialize`, le second `add_data` reprend la surcharge propriétés + heures 24/7 UTC de la Stratégie 1 (le flux n'est pas tradé, mais ses timestamps doivent être lus en UTC — le piège du 8 mars vaut pour TOUTES les données custom). Et `on_data` traite le profil **avant** le prix dans le même Slice, avec deux règles de session :

```

if self.vp in data:
    f = data[self.vp]
    if str(f["session"]) != self.session:
        # Nouvelle enchère : les niveaux SAUTENT (profil remis à zéro) -> on ne
        # détecte jamais un « franchissement » à cheval sur deux sessions.
        self.session = str(f["session"])
        self.close_prec = None
        self.lo_prec = None
    self.barres = int(float(f["barres"]))
    ...

```

⚠ Piège pythonnet rencontré en écrivant cette leçon : appeler le coefficient de lissage `self.alpha` plante `initialize` avec le message cryptique `error return without exception set` — `alpha` est une propriété .NET de `QCAAlgorithm` (le modèle alpha du framework). Les attributs de vos algos ne doivent pas écraser les membres du moteur.

La stratégie retenue (a priori) : l'acceptance

Paramètres écrits **avant** de voir la moindre courbe (Stratégie 5) : lecture `vah_break` — le close franchit le VAH **de la session en cours** par le bas : le marché accepte des prix au-dessus de la valeur de l'enchère, on suit — **confirmée par l'analyse des volumes** (EMA 24 du delta > 0 : la cassure est portée par des agresseurs acheteurs). Règles de session, a priori elles aussi : pas d'entrée hors session, ni avant la **3^e barre** d'une session (un profil d'une barre est une loterie). Le profil fournit lui-même le cadre de risque, en niveaux externes à la position (Stratégie 4) : cible = VAH + (VAH - POC), stop = VAH - 0,5 × (VAH - POC) (retomber sous le niveau conquis = hypothèse morte), edge minimal 0,2 % sur le chemin VAH → cible (≈ 2,5× l'aller-retour de frais — mesuré du **niveau franchi**, pas du prix d'entrée : le fill au close, déjà au-dessus du VAH, en mange une partie ; c'est un filtre de géométrie du profil, pas une garantie de couvrir les frais). Long/flat, `set_holdings(1.0)`, mêmes frais — comparable à tout le cours.

```

cd lean/Launcher/bin/Release
dotnet QuantConnect.Lean.Launcher.dll `
  --algorithm-language Python `
  --algorithm-type-name VolumeProfile `
  --algorithm-location "../../../../../algorithms/volume_profile.py"

```

Sortie mesurée (janvier→juin 2026 — tes chiffres évolueront avec le CSV) :

```

ENTREE 2026-01-02 17:00 [ny b3] | close=90334.0 > lo=90200 | cible=90400 stop=90100
| delta_ema=+91
SORTIE stop 2026-01-02 19:00 | close=89735.3 <= 90050

--- BILAN Volume profile (vah_break, filtre delta=True, triche=False) ---

```

Trades exécutés : 190 | frais totaux : 6950.51 \$
 Équité finale : 84319.74 \$ | rendement stratégie : -15.6803%

STATISTICS:: Net Profit -15.680% | Drawdown 15.700%
 STATISTICS:: Win Rate 31% | Total Fees \$6950.51



Trois panneaux alignés : le prix BTC/USD avec les niveaux VAH, POC et VAL du profil de session et 95 achats/95 ventes marqués à leur prix d'exécution réel ; l'EMA 24 du delta en aires vertes (acheteurs) et rouges (vendeurs) ; l'équité qui glisse de 100 000 à 84 320 \$ avec un drawdown maximal de 15,7 %.

Figure — La cassure de value area de session confirmée par le delta : -15,7 % au total. (Script : `fig_volume_profile.py`)

Verdict honnête : -15,7 %. Mieux que le B&H (-33,2 %) et que la contre-tendance nue (Stratégie 3), moins bien que la SMA (-4,3 %) : casser le haut d'une enchère de quelques heures dans un marché baissier de six mois reste un pari contre le courant de fond — le régime de la Stratégie 6 manque à l'appel.

L'analyse des volumes, mesurée

Le filtre delta sert-il vraiment ? On coupe `FILTRE_DELTA = False`, un run, même période :

vah_break (sessions)	Net	Drawdown	Ordres	Frais	Win rate
avec filtre delta (retenu)	-15,68 %	15,7 %	190	6 951 \$	31 %
sans filtre delta	-27,26 %	27,9 %	312	10 551 \$	31 %

+11,6 points de rendement, un drawdown presque divisé par deux, 3 600 \$ de frais épargnés : sur des profils de session (jeunes, nerveux), exiger que la cassure soit *portée* par les acheteurs n'est plus un raffinement, c'est la moitié de la stratégie. C'est la contribution mesurable de l'analyse des volumes : pas une stratégie en soi, un **filtre de qualité** sur les signaux de prix.

L'ancrage change tout : sessions vs fenêtre glissante

Avant d'ancrer sur les sessions, ce pipeline a d'abord été construit (et mesuré) avec un ancrage naïf : un profil **glissant des 24 dernières heures** (--zone rolling:24 du générateur). Mêmes lectures, mêmes paramètres, seul l'ancrage change :

Config (delta actif)	Ancrage glissant 24 h	Ancrage sessions
vah_break (acceptance)	-8,87 % · dd 12,3 % · 214 ordres	-15,68 % · dd 15,7 % · 190 ordres
val_reclaim (retour au POC)	+9,63 % · dd 4,0 % · WR 67 %	-2,54 % · dd 6,9 % · WR 52 %

La ligne du bas est la vraie leçon. Avec l'ancrage glissant, val_reclaim était **la première stratégie positive du cours** (+9,6 %, 67 % de gagnants) ; sur les sessions — un ancrage plus « propre » conceptuellement — **l'edge s'évapore** (-2,5 %). Un résultat qui change de signe quand on change un choix de construction *raisonnable* n'était pas un edge : c'était une **coïncidence d'échantillon** (Stratégie 5, encore). Le réflexe pro devant un backtest gagnant n'est pas « combien ? » mais « **survit-il à une perturbation anodine ?** » — ici, non.

Falsification : le look-ahead d'une heure

Toute la causalité du pipeline tient à `end_time = t+1h`. Preuve par la triche : `TRICHE_LOOKAHEAD = True` met `end_time = t` — chaque ligne de profil arrive à l'**ouverture** de la barre qu'elle résume, une heure avant d'être connaissable. Un run :

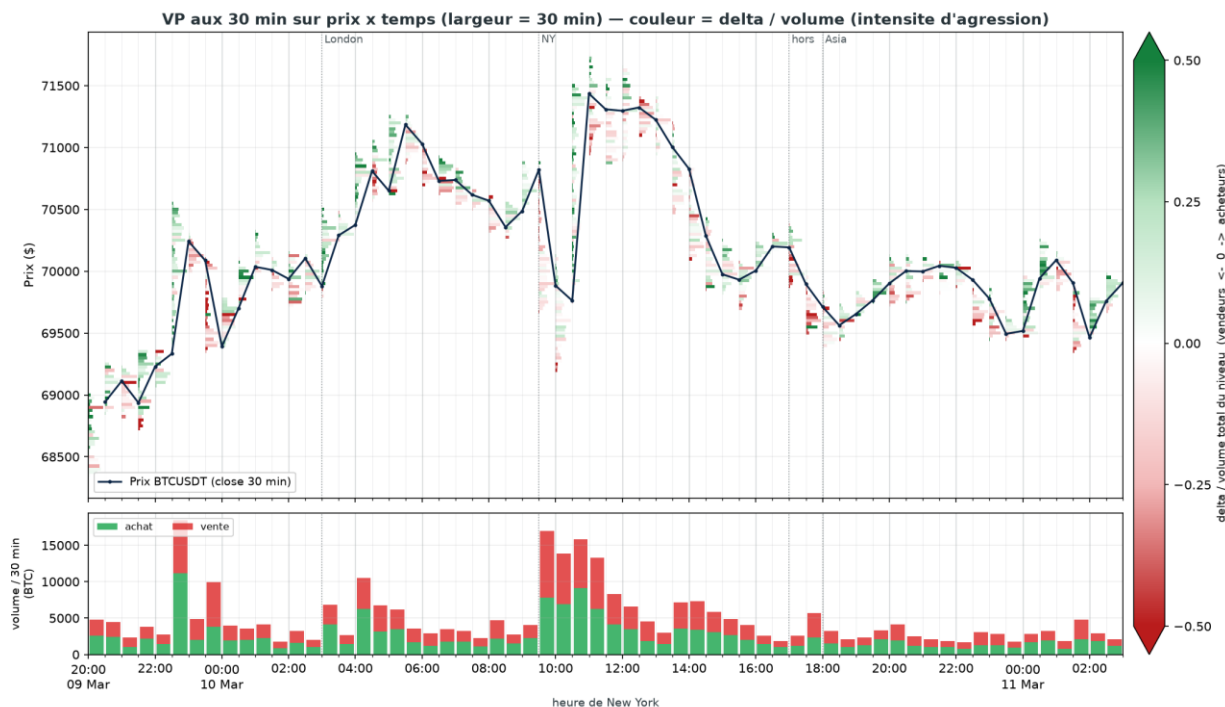
vah_break + delta (sessions)	Net	Drawdown	Ordres	Win rate
causal (<code>end_time = t+1h</code>)	-15,68 %	15,7 %	190	31 %
triche (<code>end_time = t</code>)	-20,62 %	20,7 %	144	17 %

Surprise : ici la triche fait **pire** (sur l'ancrage glissant, elle faisait *mieux* : -8,05 % contre -8,87 %). C'est le point important : un biais de futur ne « flatte » pas systématiquement le résultat — il le rend **faux**, dans un sens imprévisible qui dépend de la stratégie et des données. Un backtest contaminé n'est pas un backtest optimiste, c'est un backtest qui ne mesure plus rien. Et rien ne le signale : pas d'erreur, pas de warning — seul le run causal mis en face révèle l'écart. C'est le biais qui a réellement été trouvé (et corrigé) dans la première version de ce projet.

Coda mesurée : et si on regardait l'order-flow de plus près ?

Le filtre delta de la stratégie est **global** (l'EMA du delta de barre). On peut regarder beaucoup plus fin : découper chaque session en **VP de 30 minutes** et peindre, à chaque niveau de 25 \$, non plus le volume mais le **ratio delta/volume** — *l'intensité* d'agression, indépendante de la taille (un niveau peu traité

mais 100 % vendeur ressort autant qu'un gros niveau équilibré). On voit alors deux dynamiques : l'**absorption au sommet** (un plus-haut fait sur du rouge) et la **migration du POC**.



VP de 30 minutes sur un canvas prix × temps (heure de New York) : pour chaque créneau de 30 min, un mini-profil de largeur fixe coloré du rouge (agresseurs vendeurs) au vert (acheteurs) selon le ratio delta/volume de chaque niveau ; le prix horaire par-dessus, les ouvertures de session en pointillés, et un panneau du bas donnant le volume par 30 min décomposé achat/vente.

Figure — L'order-flow au niveau × 30 min : à l'ouverture NY, le prix fait son plus-haut sur des cases rouges (vendeurs agresseurs) = absorption. Séduisant à l'œil. (Script : `fig_vp_prix_temps.py`)

Deux descripteurs sautent aux yeux, deux hypothèses posées **a priori** : (1) le **delta au niveau franchi** — une cassure de VAH portée par des acheteurs à ce niveau ($\text{ratio_vah} > 0$) devrait battre une cassure absorbée ; (2) la **vélocité du POC** — un POC qui monte (valeur acceptée plus haut) devrait annoncer la continuation. On les mesure sur la **même issue que la stratégie** (cassure de VAH → cible avant stop), sans backtest — protocole figé : cible et stop sont mémorisés à la cassure et l'issue cherchée sur les closes suivantes, là où la stratégie, elle, sort sur des niveaux qui continuent de vivre avec la session :

```
python ../../Formations/Backtesting/LEAN/scripts/diag_descripteurs_vp.py
162 cassures de VAH avec issue résolue
```

1) DELTA AU NIVEAU VAH (ratio_vah)

cassure absorbée ($r \leq 0$)	: n= 47	win 53.2%	espérance -0.083%
cassure portée ($r > 0$)	: n=115	win 49.6%	espérance -0.150%
quartiles (rouge->vert)	: Q1 -0.068% · Q2 -0.191% · Q3 -0.175% · Q4 -0.090%		

2) VÉLOCITÉ DU POC (3 barres, n=118)

POC descend ($\text{vel} \leq 0$)	: n= 76	win 56.6%	espérance -0.083%
POC monte ($\text{vel} > 0$)	: n= 42	win 45.2%	espérance -0.239%

quartiles (baissier->hausser) : Q1 +0.064% · Q2 -0.262% · Q3 -0.063% · Q4 -0.285%

TOUTES (référence) : n=162 win 50.6% espérance -0.131%

Les deux hypothèses sont **non seulement non confirmées, mais renversées** : les cassures *absorbées* (delta rouge au VAH) font un peu mieux que les *portées*, et les cassures où le POC *descendait* battent celles où il montait. Aucune monotonie, et toutes les tranches restent à espérance négative.

⚠ Le piège est là, tentant : le Q1 de la vitesse (POC qui *chute*) sort à **+0,064 %**, 69 % de gagnants — la seule tranche positive de tout ce test. **N'y touchez pas.** n = 29, non monotone (Q2 juste à côté s'effondre), et à contre-sens de toute logique de marché. Sur 2 descripteurs × 4 quartiles = **8 comparaisons**, en trouver une positive par pur hasard est *attendu*. Y croire serait refaire l'erreur de la [Stratégie 5](#) par la porte de derrière — chercher la tranche qui gagne au lieu de tester une hypothèse.

Conclusion honnête : la dynamique d'ordre-flow qu'on voit si nettement dans ces profils ne se **convertit pas en edge mesurable** sur ces six mois — deux fois de suite, à plat voire inversée. On garde les VP 30 min comme **outil d'analyse** (lire une séance, comprendre qui pousse), pas comme signal de trading. *Voir avant de mesurer ; mesurer avant de croire.*

Ce qu'il faut retenir

- L'OHLCV **écrase** la dimension prix du volume ; le volume profile la reconstruit depuis les ticks : POC (juste prix), value area (70 % du volume), acceptance/rejet.
- **Un profil n'existe que par son ancrage.** « Journalier 00:00–23:59 UTC » est arbitraire sur un actif 24/7 ; les vraies ancrages sont les **sessions** (en heure de New York — DST !), chacune une enchère autonome dont le profil se **développe** barre après barre (sous-VP).
- Le **delta** (agresseurs acheteurs – vendeurs) est un filtre de qualité mesurable : sur les profils de session, +11,6 points de rendement sur les mêmes signaux.
- Un edge qui **change de signe quand l'ancrage change** (+9,6 % → -2,5 %) n'était pas un edge : tester la robustesse d'un résultat à ses choix de construction AVANT d'y croire.
- **La causalité des features est un contrat d'horodatage** : la ligne t n'existe qu'à $t+1h$; et un look-ahead ne rend pas un backtest « meilleur », il le rend **faux** — dans un sens imprévisible (-4,9 pts ici, +0,8 pt sur l'ancrage glissant).
- Le plus **visible** n'est pas le plus **mesurable** : le delta au niveau franchi et la vitesse du POC, éclatants à l'œil, sortent à **plat voire inversés** (coda) ; et une tranche gagnante sur 8 comparaisons est un mirage de sur-apprentissage, pas un edge.
- Les attributs de vos algos peuvent **écraser des membres .NET** de QCAAlgorithm (self.alpha → crash cryptique) : préfixe ou nom français, et lire le message deux fois.

Précédent : [Stratégie 6 — La stratégie avancée](#) · Cette leçon étend la section avec la famille order-flow ; la parité de ces stratégies côté vectorbt (section 02) devra transporter les features (features_vp.csv) à l'identique — même CSV, même contrat d'horodatage.